Proof-to-Byte: Assumption-Minimal, Cross-Substrate Binding Assurance for PQ/T Hybrid KEMs

Ziang Li

***Abstract*—Post-quantum hybrid key-encapsulation (PQ/T KEMs, combining a lattice KEM with an elliptic-curve KEM) is being deployed across TLS, Signal, and MLS faster than its *deployment safety* can be assured: the same period has seen side-channel breaks (KyberSlash), binding / re-encapsulation breaks (PQXDH; the ML-KEM malicious-key binding failures of Schmieg), and a widening gap between what a hybrid's *specification* guarantees and what its compiled, multi-platform *implementation* actually does. We present Q-Periapt, an assurance suite for PQ/T hybrid KEMs organised around one principle: *a security property is only as trustworthy as its weakest realization*. Its combiner, ContextBound, achieves binding (MAL-BIND-K-CT and MAL-BIND-K-PK) reducing to collision-resistance of SHA3 *alone*—with no binding assumption on the component KEMs—and we machine-check this in EasyCrypt. The injective encoding is a proved lemma (not an axiom), and—stated honestly—the result is in substance a *commitment / transcript-binding* theorem (the session key commits to the full transcript; the component decapsulation is inert in the argument, which is precisely why no KEM assumption is needed), instantiated in the Cremers–Dax–Medinger Figure 6 game for both rejection styles. Crucially, the *same* proven combiner is realized byte-identically across five language faces and three operating systems—executed natively on three instruction-set architectures and cross-compiled, with by-construction identity, on two more—verified by a six-method conformance matrix and a self-validating source→binary constant-time probe (configured as a CI gate; measured locally) that we show *discriminates* a clean primitive (libcrux ML-KEM: zero secret-dependent branches after compilation) from a leaky one (HQC's PQClean constant-weight sampler). We integrate the combiner into a production TLS 1.3 path (a rustls `CryptoProvider`) and evaluate handshake tail latency under real `tc netem`. We are explicit about what is and is not novel: the binding *fact* is a known consequence of recent results; our contribution is the *conjunction*—an assumption-minimal, machine-checked binding result whose proven object is demonstrably identical across a heterogeneous deployment surface, with reproducible gates that catch the failure classes that have broken deployed hybrids.**

***Index Terms*—Post-quantum cryptography, hybrid key encapsulation, binding/committing security, formal verification, constant-time, cross-platform assurance, TLS.**

I. Introduction

THE migration to post-quantum (PQ) key establishment is being done conservatively, through *hybrids*: a PQ KEM (ML-KEM [1]) run alongside a classical KEM (X25519), with the two shared secrets combined so that the session key is secure if *either* component holds. Hybrid KEMs are now shipping in TLS 1.3 (`X25519MLKEM768`), in Signal's PQXDH, and in MLS, and are specified in CFRG drafts such as X-Wing [5].

Author affiliation omitted for review. Artifact (open source): https://github.com/billlza/q-periapt.

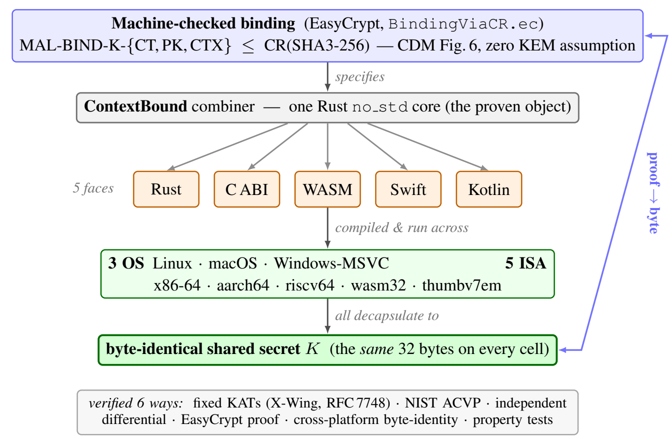

Fig. 1. Proof-to-byte. One machine-checked binding result (top) is realized by a single Rust `no_std` core, exposed through five language faces and run across three operating systems and five instruction-set architectures, all converging on a byte-identical shared secret K (bottom); verified six ways.

The gap this paper addresses: Hybrids are being deployed faster than their *deployment safety* can be assured. In the same window, the community has seen (i) *side-channel* breaks of PQ implementations (KyberSlash [6]); (ii) *binding* failures—the PQXDH re-encapsulation issue, and Schmieg's proof that ML-KEM in its FIPS-203 *expanded* decapsulation-key form is neither MAL-BIND-K-CT nor MAL-BIND-K-PK [3]; and (iii) a widening gap between a hybrid's *specification*, its formally-verified source, and the *compiled artifact* that actually runs—across the many languages, operating systems, and instruction sets that real deployments target. This safety question is distinct from, and orthogonal to, *auditability* and *crypto-agility* (which describe *what* crypto is running): it asks whether the running hybrid is actually *safe*.

Approach: We present Q-Periapt, an assurance suite for PQ/T hybrid KEMs built around one principle: a security property is only as trustworthy as its weakest realization. Figure 1 is the organising idea—a single machine-checked binding result whose proven object is realized byte-identically across the whole deployment surface ("proof-to-byte"), guarded by reproducible gates (configured in CI; we report constant-time results here as local runs) that target the specific failure classes above.

Contributions:

- **C1 (Section III).** An *assumption-minimal, machine-checked commitment / transcript-binding* result: ContextBound's session key commits to the full transcript, reducing MAL-BIND-K-CT, -K-PK, and a (non-standard) context extension -K-CTX to collision-resistance of SHA3 *alone*, with the injective encoding proved and the Cremers–Dax–Medinger Figure-6 game discharged for both rejection styles in EasyCrypt. We are explicit (Section III) that the cryptographic content is this injective-encoding + CR(SHA3) argument—the component Decaps is *inert*, which is exactly why no KEM assumption is needed—and that C1's value is its completeness, mechanization, and role as the proven object tied to C2, not proof difficulty.

- **C2 (Section IV).** *Proof-to-byte cross-substrate equivalence*: the same proven combiner is byte-identical across 5 language faces and 3 OSes, executed natively on 3 ISAs and cross-compiled with by-construction identity on 2 more, verified by a six-method conformance matrix.
- **C3 (Section V).** Reproducible assurance *instruments*, configured as CI gates: a self-validating source→binary constant-time probe that *discriminates* clean (ML-KEM) from leaky (HQC) (HQC's leak is known; the contribution is the discriminating, gated oracle); an executable *regression oracle* for the known dk-format binding coupling; and two independent symbolic provers. We report the constant-time measurements honestly as local runs.
- **C4 (Section VI).** A production TLS 1.3 path (rustls `CryptoProvider`), evaluated under real `tc netem`.

What is and is not novel: We state the boundary plainly. *Not novel, and attributed:* that a "hash-everything" combiner is binding from collision-resistance alone while a lean combiner inherits the omitted component's binding is the Chempat result [4]; ContextBound *is* that construction, not a new primitive. That ML-KEM's expanded-dk form breaks binding (and the seed form fixes it) is Schmieg's [3]. The binding-notion lattice is CDM's [2]. That a constant-time harness must mark only secret sub-fields is standard practice [6]. *Novel:* the *conjunction* C1∧C2—a CR-only, machine-checked binding result whose proven object is demonstrably byte-identical across a heterogeneous substrate, which no prior work provides; the self-validating source→binary *discriminator* as a reusable CI artifact; and the operationalized design invariant that binding the ciphertext and public key in the KDF makes hybrid binding robust to the component KEM's key-serialization format. We do **not** claim stronger binding than X-Wing (Section III), nor any live exploited deployment. Table I makes the conjunction concrete: the delta is that no prior effort ties a mechanized hybrid-combiner binding theorem to a byte-identical multi-substrate realization with source→binary and TLS gates—and we are equally explicit that an axis we do *not* hold (a verified specification-to-binary implementation) is held by formosa-mlkem.

TABLE I

POSITIONING. ✓ HOLDS; ∼ PARTIAL; BLANK: NOT PROVIDED. THE CONTRIBUTION IS THE *conjunction* (LAST COLUMN), NOT ANY SINGLE ROW.

	X-Wing	formosa	SandboxAQ	Chempat	**this work**
Mechanized binding (combiner)			∼		✓
Both rejection styles, $K \neq \perp$				∼	✓
Zero KEM-binding assumption				✓	✓
Byte-identical multi-substrate object					✓
Source→binary CT gate		∼			✓
Production TLS 1.3 path	∼				✓
Verified spec→binary impl. *(we do not)*		✓	∼		

II. BACKGROUND AND THREAT MODEL

A. PQ/T hybrid KEMs and combiners

ML-KEM [1] is the standardized form of CRYSTALS-Kyber [11]; like most lattice KEMs it applies a Fujisaki–Okamoto transform [13], [14] with *implicit rejection* (a malformed ciphertext yields a pseudorandom key rather than $\perp$). The companion signature standards are ML-DSA [9] and SLH-DSA [10], and the broader process is surveyed in [12]. Hybrid key exchange in TLS 1.3 [16] follows the IETF design [17]—e.g. the `X25519MLKEM768` group [18] over X25519 [15]—and deployed PQ/T protocols include Signal's PQXDH [19] and Apple's PQ3 [20].

A hybrid KEM combines component shared secrets ss_{pq} and ss_{tr} into a session key K. The design space ranges from simple *concatenation* KDFs (`X25519MLKEM768`) to combiners that additionally absorb ciphertexts and public keys. X-Wing [5] uses a fixed "lean" combiner $K = \mathrm{SHA3}(ss_{\mathrm{pq}} \,\|\, ss_{\mathrm{tr}} \,\|\, ct_{\mathrm{tr}} \,\|\, pk_{\mathrm{tr}} \,\|\, \mathrm{label})$, omitting the ML-KEM ciphertext and public key.

B. KEM binding

We use the Cremers–Dax–Medinger (CDM) X-BIND-P-Q framework [2]. A *transcript* is a tuple (pk, ct, K) produced by a (possibly malicious) execution; the two "observable" quantities a binding notion may constrain are the public key PK, the ciphertext CT, and the derived key K. For $P, Q \in \{K, \mathsf{PK}, \mathsf{CT}\}$, a KEM is X-BIND-$P$-$Q$ if no adversary in the game below wins:

> **Game** $\mathsf{Bind}_{\mathcal{A}}^{P,Q}$: the adversary $\mathcal{A}$ outputs two transcripts (pk_0, ct_0, K_0) and (pk_1, ct_1, K_1). $\mathcal{A}$ *wins* iff $P_0 = P_1 \;\wedge\; Q_0 \neq Q_1 \;\wedge\; K_0 \neq \perp \;\wedge\; K_1 \neq \perp$.

Three adversary classes parameterize how the key material in each transcript is produced, in increasing strength: **HON** (honest KeyGen), **LEAK** (honest keys, secret keys leaked to $\mathcal{A}$), and **MAL** (adversarially generated keys). Thus MAL $\supseteq$ LEAK $\supseteq$ HON, and CDM prove monotonicity across the (P, Q) lattice, so MAL-BIND-K-CT and MAL-BIND-K-PK jointly yield the whole K-binds-{PK,CT} sub-lattice. For implicitly-rejecting KEMs such as ML-KEM, these two are the achievable ceiling on those axes; the dual X-BIND-CT-$*$ notions (a ciphertext binding the key or public key) are *structurally unachievable*, since decapsulation never returns $\perp$ and a mutated ciphertext always yields *some* key. The binding of implicitly-rejecting KEMs—and how it fails under malicious, expanded-form keys—is studied in detail by Schmieg and others [3], [21].

Algorithm 1: ContextBound combiner. `CompatXWing` omits $ct_{\mathrm{pq}}, pk_{\mathrm{pq}}, S, v, x$ and uses X-Wing's label.

Input: label L, suite id S, version v, secrets $ss_{\mathrm{pq}}, ss_{\mathrm{tr}}$, ciphertexts $ct_{\mathrm{pq}}, ct_{\mathrm{tr}}$, public keys $pk_{\mathrm{pq}}, pk_{\mathrm{tr}}$, context x
Output: 32-byte shared secret K
$\mathsf{lp}(f) \leftarrow \mathsf{be8}(|f|) \,\|\, f$ `// 8-byte big-endian length prefix`
$T \leftarrow [\,L,\ S,\ \mathsf{be4}(v),\ ss_{\mathrm{pq}},\ ss_{\mathrm{tr}},\ ct_{\mathrm{pq}},\ pk_{\mathrm{pq}},\ ct_{\mathrm{tr}},\ pk_{\mathrm{tr}},\ x\,]$
$e \leftarrow \mathsf{lp}(T_0) \,\|\, \mathsf{lp}(T_1) \,\|\, \cdots \,\|\, \mathsf{lp}(T_{n-1})$ `// injective encode`
return SHA3-256(e)

C. Threat model

We consider three adversaries. The *malicious-key (MAL) binding* adversary supplies adversarial keypairs and seeks colliding transcripts (Section III). The *binary-level constant-time* adversary observes secret-dependent control flow / memory indexing in the *compiled* code (Section V). The *downgrade/agility* adversary attacks profile and policy negotiation (handled by fail-closed policy binding; out of scope here). Confidentiality of the component primitives (IND-CCA2 of ML-KEM, the Diffie–Hellman assumption for X25519) is assumed, not reproved.

III. ContextBound and the Machine-Checked Kernel

A. Construction

ContextBound derives $K = \mathrm{SHA3\text{-}256}\big(\mathrm{encode}[\mathsf{LABEL}, \mathsf{suite\_id}, \mathsf{policy\_version}, ss_{\mathrm{pq}}, ss_{\mathrm{tr}}, ct_{\mathrm{pq}}, pk_{\mathrm{pq}}, ct_{\mathrm{tr}}, pk_{\mathrm{tr}}, \mathsf{context}]\big)$, where `encode` concatenates each field under a fixed-width 8-byte big-endian length prefix (Algorithm 1). Two design choices are load-bearing. *(i) Injective, length-prefixed encoding.* A naive concatenation $a \,\|\, b$ is ambiguous—("ab", "") and ("a", "b") collide—which is precisely the structure a binding adversary exploits. Prefixing every field with its fixed-width length makes the encoding self-delimiting and therefore injective (proved, Section III), so distinct transcripts map to distinct byte strings and the binding question reduces cleanly to collision-resistance of the hash. *(ii) Absorb everything.* ContextBound feeds *all* component ciphertexts and public keys (and an application context, and a domain-separating label, suite identifier, and policy version) into the hash. By the combiner-binding theorem of Chempat [4], the binding advantage is then bounded by $\mathrm{Adv}^{\mathrm{CR}}$ alone, with no residual term requiring any component KEM to be binding—trading a KEM-self-binding assumption for a single, well-studied collision-resistance assumption. The dual profile `CompatXWing` reproduces X-Wing's lean combiner byte-for-byte for interoperability; `ContextBound` is the hash-everything profile and is policy-selected when an application needs context binding or wishes to avoid the serialization-format dependence of Section V. The suite is open source.[1]

[1] https://github.com/billlza/q-periapt

B. The EasyCrypt development

Figure 2 shows the mechanized reduction. The encoding's injectivity, `encode_inj`, is a *proved lemma*, not an assumed one. The proof has two steps. A field-level lemma, `lp_cat_inj`, shows that a single length-prefixed field is self-delimiting: if $\mathsf{lp}(a) \,\|\, x = \mathsf{lp}(b) \,\|\, y$ then $a = b$ and $x = y$, because the two 8-byte prefixes have equal width and (being injective) force $|a| = |b|$, after which list-concatenation cancellation (`eqseq_cat`) splits off equal field bodies and equal remainders. A structural induction over the field list then lifts this to whole encodings: a non-empty encoding has size ≥ 8 and so can never equal the empty one, ruling out boundary-shift collisions, and the inductive step peels one field with `lp_cat_inj`. The whole argument bottoms out at two elementary facts about the length field—that an 8-byte big-endian integer occupies 8 bytes (`be8_size`) and is injective (`be8_inj`). On top of `encode_inj` we discharge (i) a generic transcript-collision reduction `bind_le_cr`: a `byequiv` argument that maps any adversary winning the binding game to one finding an H-collision, since a differing observable forces a differing field list (hence, by injectivity, a differing encoding) while the colliding key forces equal hashes; and (ii) the *KEM-aware* game, in which the MAL adversary supplies the keypairs and K is *derived* via the component Decaps functions and the combiner. We mechanize both the implicit-rejection specialization (`malbind_*_le_cr`) and the fully general CDM Figure-6 game with explicit rejection (`malbind_*_xrej_le_cr`), where the $K \neq \perp$ side condition is *present and load-bearing*—removing it makes the proof fail. Every theorem reduces only to $\mathrm{CR}(\mathrm{SHA3})$; the development compiles under the EasyCrypt checker (we pin the checker and SMT-solver versions in the artifact), and each theorem is verified load-bearing on `encode_inj` by an adversarial negative control.

What this is, precisely. We state the cryptographic content plainly, because a CDM-literate reader should not over-read the label. Since K *is defined* as $H(\mathrm{encode}[\ldots])$ and the tuple contains the ciphertexts and public keys *directly*, the proof is in substance a *commitment / transcript-binding* statement: it shows that one cannot collide K while changing a ciphertext, public key, or context, by reducing to $\mathrm{CR}(H)$ over the proved injective encoding. The component Decaps—and hence the adversary's freedom over key material that CDM binding is fundamentally about—is *inert* in the argument: the theorems would hold for *any* Decaps, which is exactly what "zero KEM assumption" means and why it is achievable. The value of C1 is therefore its *completeness* (both rejection styles, with $K \neq \perp$ discharged), its *mechanization*, and its role as the *proven object* carried, byte-identically, across the substrate of Section IV—not proof difficulty. The concrete attack-resistance arguments (re-encapsulation, the dk-format coupling of Section V) are *design arguments* consistent with this commitment property, not claims the EasyCrypt artifact witnesses at the Decaps level.

C. Honest binding position

Figure 3 states our position precisely. On the standard {CT,PK} axes, ContextBound and a correctly-implemented

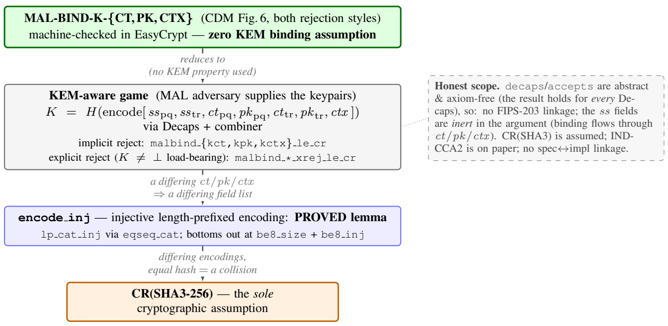

Fig. 2. The kernel as a reduction tower: MAL-BIND-K-{CT,PK,CTX} (CDM Figure 6, both rejection styles) reduces—using no property of Decaps—through the proved `encode_inj` to collision-resistance of SHA3, the sole cryptographic assumption.

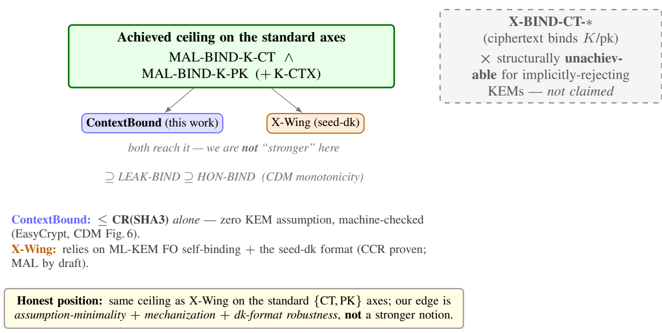

Fig. 3. Honest binding position. Both schemes reach the standard ceiling; our edge is assumption-minimality, mechanization, and dk-format robustness—not a stronger notion. X-BIND-CT-$*$ is structurally unachievable for implicitly-rejecting KEMs and is not claimed.

seed-dk X-Wing both reach the MAL-BIND-K-{CT,PK} ceiling; we are *not* “stronger” there. Our edge is (a) *assumption-minimality*: ContextBound’s binding reduces to CR(SHA3) alone, whereas X-Wing’s relies on ML-KEM’s Fujisaki–Okamoto self-binding together with the seed-dk format; (b) *mechanization*; and (c) *dk-format robustness* (Section V). Honest scope of the mechanization: Decaps is modelled as an abstract function (which is exactly why “zero KEM assumption” is literal), so there is no FIPS-203 linkage and the shared-secret fields are inert in the binding argument; CR(SHA3) is assumed; IND-CCA2 robustness is argued on paper; and there is no specification-to-implementation linkage proof.

IV. Proof-to-Byte Cross-Substrate Realization

A. Architecture

The combiner and its encoding live in a single dependency-free Rust `no_std` core; the component primitives are *injected* through narrow traits (a `Kem` trait for ML-KEM and X25519, an extendable-output `Xof` trait for the hash), so the proven object—the encoding and combiner—is the same code regardless of which vetted backend (libcrux ML-KEM, x25519-dalek, libcrux SHA3) is plugged in. This trait boundary is also where *crypto-agility* lives: a `Kem`’s `C2PRI` flag (ciphertext second-preimage resistance) gates which combiner profiles it may use, and a signed, versioned *policy* object selects the suite and profile and binds a downgrade-resistant `policy_version` into the transcript (Algorithm 1). Shared secrets are zeroized on drop, and the composition code is written in a branchless, mask-based constant-time style (Section V).

The core is exposed through five language faces—Rust, a C ABI (via `cbindgen`), WebAssembly (via `wasm-bindgen`), Swift (over the C ABI), and Kotlin (via the Java Foreign Function & Memory API)—each a thin shim that marshals bytes in and out of the same core. Because the proven object is one piece of code and every face is a marshalling layer over it, the cross-substrate question reduces to a single, checkable property: does *every* realization produce the same bytes?

B. The six-method conformance matrix

Table II lists the six orthogonal methods (NIST ACVP vectors [31] among them) that answer this. They are deliberately heterogeneous in their oracle and in what they catch, so a defect that escapes one is caught by another. Table III reports the substrate coverage with honest per-cell status: the byte-identical shared secret is *run* natively on x86-64, aarch64, and wasm32; on riscv64 and the bare-metal `thumbv7em` target the core cross-compiles cleanly and byte-identity follows by construction from the FIPS-deterministic encodings and identical source, but is not executed on a native runner. This honest accounting is what makes the cross-substrate claim defensible.

Footprint. The deployable surface is small: the stripped C-ABI shared library—the *entire* suite, including libcrux ML-KEM and ML-DSA—is 441 KB; the WebAssembly module is 249 KB; and the dependency-free core cross-compiles to the bare-metal `thumbv7em` target, so the same proven combiner runs from a server down to an embedded MCU.

V. CI-Gated Assurance

A. *A source→binary constant-time* discriminator

libcrux proves its ML-KEM secret-independent at the *source* level (a typed discipline, machine-checked). The orthogonal *binary*-level question—does the compiler reintroduce a secret-dependent branch on the genuine secret path?—is answered by a probe that marks only the genuinely-secret decapsulation-key sub-fields ($\hat{s}$, z) and runs the real libcrux `decapsulate` under Memcheck. The probe is *self-validating*: a planted secret-indexed load is caught (harness sanity), and marking the embedded *public* key flags 5696 reports (Memcheck demonstrably sees real branches in this binary). With only the genuine secret marked, the probe reports **zero** flags on aarch64: source-level secret-independence survives compilation. Run against the suite’s unaudited HQC backend, the same probe flags **193** reports localized to `vect_set_random_fixed_weight`—so the probe is a *discriminator* (Fig. 4): clean ML-KEM versus leaky HQC in one framework. (HQC’s leak is known; the contribution is the discriminating, gated artifact and the corollary that per-backend constant-time gating is necessary, not decorative.)

Why marking granularity matters. A naive harness that marks the *whole* serialized decapsulation key secret produces, on the NEON path, 2848 reports across 30 branches comparing 12-bit coefficients to q—which look exactly like a KyberSlash-class leak. They are not: the FIPS-203 `dk` *embeds* the public key

TABLE II
THE SIX ORTHOGONAL VERIFICATION METHODS.

Method	Reference / oracle	Independence	What it catches
Fixed KATs	X-Wing draft vectors; RFC 7748	published vectors	combiner and X25519 spec-conformance regressions
NIST ACVP	*usnistgov* ACVP, full FIPS family	NIST ground truth	primitive non-conformance (FIPS 203/204/205)
Multi-backend differential	RustCrypto `ml-kem`/`ml-dsa`; orion X25519	independent re-implementations	implementation bugs (output must stay byte-identical)
EasyCrypt proof	machine-checked; CR(SHA3)	formal (computational)	binding-design flaws in the combiner
Cross-platform byte-identity	shared test vector	5 faces × substrates	per-substrate divergence (same K everywhere)
Generative property tests	proptest: 7 properties × 128 cases	randomized	edge-case violations (injectivity, domain separation, K-CTX)

TABLE III
CROSS-SUBSTRATE COVERAGE (✓ VERIFIED, CI OR LOCAL).

(a) ISA targets (Rust core; the byte-level claim)

ISA	runner	build	byte-id. K	binary-CT
x86-64	Linux, Win-MSVC	✓	✓ (run)	✓ Memcheck
aarch64	Linux, macOS	✓	✓ (run)	✓ Memcheck
wasm32	node	✓	✓ (run)	source-CT
riscv64	Linux (cross)	✓	by constr.†	source-CT
thumbv7em	bare-metal	✓	by constr.†	n/a

† cross-compiled, not run natively; identity follows from FIPS-deterministic encodings + identical source.

(b) language face × OS (shared vector → same K)

Face	Linux	macOS	Windows
Rust	✓	✓	✓
C ABI	✓	✓	✓ (MSVC)
WASM (node)	✓	✓	✓
Swift	–	✓	–
Kotlin (JVM/FFM)	✓	✓	–

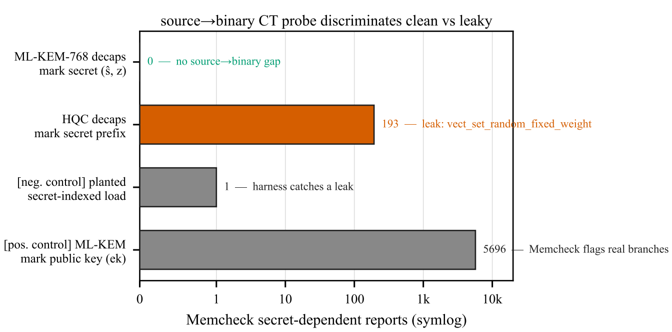

Fig. 4. The source→binary CT probe discriminates: the genuine ML-KEM secret yields zero flags; the same harness flags HQC's constant-weight sampler (193) and validates itself with two controls.

($\mathtt{dk} = \mathtt{dk\_pke} \,\|\, \mathtt{ek} \,\|\, H(\mathtt{ek}) \,\|\, z$), and those branches are the compiler's scalar lowering of the public-key reduction run during FO re-encryption—operating on *public* bytes. Marking only the genuinely-secret sub-fields ($\hat{s}$ and the implicit-rejection seed z) yields zero reports. This is standard practice [6], but the embedded-public-key pitfall is a real trap; our probe encodes the correct granularity and gates it.

B. Binding is contingent on the component dk format

We demonstrate, executably against real libcrux, that a lean (X-Wing-shaped) combiner's MAL-BIND-K-PK is *contingent on* the component KEM's key-serialization format. Schmieg's construction exploits that the FIPS-203 *expanded* `dk` stores the implicit-rejection seed z as a substitutable field: an adversary equalizes z across two otherwise-distinct keypairs, so a garbage ciphertext implicit-rejects to the *same* $J(z \,\|\, c)$ under two different public keys. The lean combiner—which omits pk_{pq}—then yields the same hybrid key for $\mathtt{ek}_0 \neq \mathtt{ek}_1$, breaking MAL-BIND-K-PK; ContextBound, which absorbs pk_{pq}, does not. We stress that this is *not* a break of X-Wing, which mandates the seed-dk format that defeats the attack (KeyGen re-derives z); it is a robustness separation of the combiner *shape*, and the operationalized lesson that binding ct/pk in the KDF removes a latent dependence on the component library's serialization choice—a dependence an implementation that caches expanded keys for performance could silently reintroduce. Our test executes this against real libcrux as a CI-configured regression oracle.

C. Multi-prover assurance

Beyond the computational EasyCrypt proof, the server-authenticated hybrid handshake is modelled in two independent symbolic provers. The Tamarin model captures the ML-KEM and X25519 key exchanges, the combiner as an opaque one-way function, and an ML-DSA server signature, with rules that let the adversary compromise either KEM or reveal the signing key; it proves executability, server authentication, and—the headline lemma—*hybrid robustness*: under an honest server identity the session key remains secret unless *both* KEMs are broken. A ProVerif model establishes the analogous queries under a different abstraction (the classical leg as a second idealized KEM), giving an independent cross-check. All three formal artifacts are configured as CI gates (EasyCrypt `make check`, Tamarin, ProVerif), with checker and solver versions pinned in the artifact.

VI. PRODUCTION INTEGRATION AND EVALUATION

A. Microbenchmarks

Table IV reports primitive and hybrid costs (Criterion, point estimates, on an Apple-Silicon arm64 host; the artifact

TABLE IV
PRIMITIVE AND HYBRID COST (μS) AND KEY/CIPHERTEXT SIZES (BYTES). ARM64 HOST.

	time (μs)			size (B)		
Scheme	keygen	encaps	decaps	ek	dk	ct
ML-KEM-512	6.6	6.1	6.8	800	1632	768
ML-KEM-768	11.3	11.0	11.8	1184	2400	1088
ML-KEM-1024	18.5	16.5	16.7	1568	3168	1568
X25519	44.1	89.6	45.0	32	32	32
Hybrid, ContextBound	—	108.4	65.4	1216	—	1120
Hybrid, CompatXWing	—	102.2	59.2	1216	—	1120

reproduces them and an x86-64 run). Two findings matter for deployment. First, *the post-quantum leg is not the bottleneck*: ML-KEM-768 encapsulation is $11.0\,\mu$s, roughly $8\times$ cheaper than X25519 encapsulation ($89.6\,\mu$s, an ephemeral keygen plus a Diffie–Hellman), so the classical half dominates hybrid CPU—a useful corrective to the assumption that PQ is the expensive part. Second, *the hash-everything combiner is cheap*: ContextBound costs only $+6.2\,\mu$s over the byte-exact X-Wing combiner CompatXWing (encapsulation 108.4 vs. $102.2\,\mu$s; decapsulation 65.4 vs. $59.2\,\mu$s)—the price of binding the full transcript is sub-$10\,\mu$s, well under the $\sim 190\,\mu$s X25519 cost and far below any network RTT.

B. Handshake latency under emulated networks

We expose ContextBound (and CompatXWing) as a TLS 1.3 key-exchange group via a rustls [32] `CryptoProvider`, using private-use group codes; a real loopback TLS 1.3 handshake completes over this provider. We evaluate handshake *time-to-session* (TCP connect plus full TLS 1.3 handshake) over a real loopback socket shaped by `tc netem`, comparing a classical X25519 baseline against the two PQ/T profiles. Figure 5 reports the result: the three curves coincide because latency is RTT-dominated, and the PQ/T overhead is a *fixed* cost ($\sim 190\,\mu$s of ML-KEM CPU plus the wire bytes of Fig. 6). At realistic RTT this is negligible: at RTT ≥ 20 ms the measured overhead is *within run-to-run noise*—its sign varies across repeated runs (Fig. 5b, error bars span two repetitions)—while the only reproducible cost is the $\sim 190\,\mu$s of CPU visible at RTT 0. The hybrid's $+2.27$ KB (one ML-KEM-768 keyshare each way) fits within the existing TLS flights and triggers no additional round-trip. The two combiner profiles are indistinguishable: the stronger binding of the hash-everything combiner is free in latency. We report these measurements honestly as obtained on a virtualized host; a turnkey script reproduces the protocol on quiesced bare-metal hardware for the camera-ready (the RTT-dominated points are already stable across repetitions).

Under jitter and loss. Adding 3 ms of jitter leaves the picture unchanged (all three groups ≈ 41 ms median, ≈ 50 ms p99.9). Under 1–3% packet loss the median is likewise unaffected—loss is rare relative to a handshake—but the *tail* jumps to ~ 1 s, dominated by a TCP retransmission timeout per lost packet rather than by crypto or wire size; this RTO-driven tail falls on the classical and PQ/T handshakes alike, and at our sample size the loss-event noise precludes a precise per-group tail comparison. The honest takeaway is that the hybrid's extra keyshare does not *systematically* worsen loss resilience: the median is byte-insensitive and the tail is governed by transport, not by the combiner.

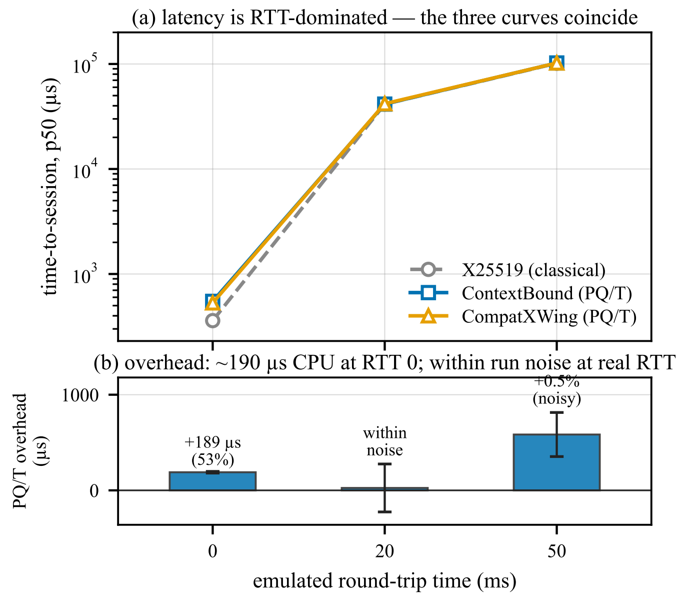

Fig. 5. Time-to-session under real `tc netem` (mean of two repetitions). (a) latency is RTT-dominated—the three curves coincide. (b) the overhead is $\sim 190\,\mu$s of CPU at RTT 0; at realistic RTT it is within run-to-run noise (error bars span the two repetitions and cross zero at RTT 20 ms).

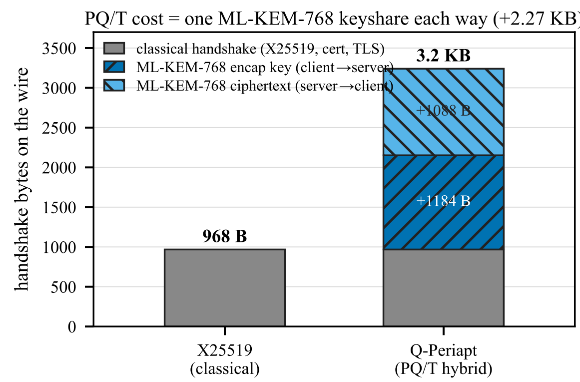

Fig. 6. Handshake wire budget. The PQ/T cost is one ML-KEM-768 keyshare each way (+2.27 KB), which fits inside the existing flights.

VII. DISCUSSION

When to use which profile: `CompatXWing` exists for interoperability: it is byte-exact X-Wing and should be selected when talking to an X-Wing peer. `ContextBound` is the default for first-party deployments and is preferable whenever (i) the application has a meaningful context (a protocol/role/version label, a channel binding) that the session key should commit to; or (ii) the deployment cannot guarantee that its component KEM will only ever be instantiated in the seed-dk form—in which case binding $pk_{\rm pq}$ and $ct_{\rm pq}$ directly removes the latent serialization dependence of Section V. The cost of that robustness, measured in Section VI, is sub-$10\,\mu$s and invisible end-to-end.

The assurance philosophy: Our position is deliberately modest about cryptographic novelty and deliberately ambitious about *dependability*: a hybrid is trustworthy only to the extent that its weakest realization is. The recurring failures of the last two years—KyberSlash, PQXDH, the ML-KEM malicious-key binding gaps—were not failures of the underlying hard problems but of *realizations*: a compiled branch, a serialization format, a missing transcript field. The contribution of this paper is a discipline that makes those realization-level properties *checkable and reproducible*: a binding result whose proven object is carried byte-for-byte across the substrate, a constant-time probe that discriminates clean from leaky, and a regression oracle for the key-format coupling. We expect the individual instruments to be reused independently of ContextBound.

Generality: Nothing in the approach is specific to ML-KEM + X25519. The combiner and its proof are generic over the component KEMs; the trait-injected backends already include the full ML-KEM and ML-DSA families and an (unaudited) HQC backend used here as the CT discriminator's leaky reference. As new PQ primitives are standardized, the same proof-to-byte discipline applies unchanged.

VIII. Related Work

KEM binding. CDM [2] introduced the X-BIND-P-Q lattice and monotonicity; Schmieg [3] proved ML-KEM's expanded-dk binding failures; Chempat [4] established that hash-everything combiners are binding from collision-resistance while lean combiners inherit the omitted component's binding (ContextBound is an instance). Context binding is the KEM-layer analogue of context-committing AEAD [7]; the role of public contexts in hybrid KEMs is analysed in [8]. **Side channels.** KyberSlash [6] and the ctgrind/TIMECOP discipline motivate our binary-CT gate; libcrux/HACL* provide source-level guarantees we corroborate and discriminate against. **Hybrids in deployment.** X-Wing [5] and the TLS `X25519MLKEM768` group are the deployed baselines we position against. **Formal PQ.** formosa-mlkem [24] mechanizes ML-KEM IND-CCA; the SandboxAQ EasyCrypt-KEMs library mechanizes LEAK-BIND-K-PK; our EasyCrypt [25] development adds the (commitment-style) MAL Figure-6 instantiation for the hybrid combiner, complemented by symbolic models in Tamarin [26] and ProVerif [27].

IX. Limitations

The mechanized binding result is over an *abstract* Decaps: it is exactly the "zero KEM assumption" that makes the result general, but it carries no FIPS-203 linkage, the shared-secret fields are inert in the argument, and CR(SHA3) and IND-CCA2 remain assumptions argued respectively in the model and on paper; there is no specification-to-implementation linkage proof. Binary-level constant-time tooling is mature only on x86-64 and aarch64; riscv64 and wasm32 are covered at source level plus upstream attestation. ACVP byte-identity is conformance, not CMVP certification. The suite is research-grade and has not had an independent third-party audit. The netem evaluation was obtained on a virtualized host; bare-metal reproduction is provided as a script.

X. Conclusion

A PQ/T hybrid is only as safe as its weakest realization. We closed that gap with an assumption-minimal, machine-checked binding result whose proven object is byte-identical across a heterogeneous deployment surface, guarded by reproducible gates (configured in CI)—a source→binary constant-time discriminator, a binding key-format coupling demonstration, and multi-prover symbolic models—and validated on a production TLS 1.3 path. The genuinely new element is the conjunction, not any single fact; we have been explicit about the boundary throughout.

References

[1] National Institute of Standards and Technology, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard," 2024.
[2] C. Cremers, A. Dax, and N. Medinger, "Keeping Up with the KEMs: Binding Security of Key Encapsulation Mechanisms," *ACM CCS*, 2024. (ePrint 2023/1933.)
[3] S. Schmieg, "Unbindable Kemmy Schmidt: ML-KEM is neither MAL-BIND-K-CT nor MAL-BIND-K-PK," IACR ePrint 2024/523, 2024.
[4] J. Güneysu, K. Hövelmanns, *et al.*, "Binding Security of Combined KEMs / Chempat," IACR ePrint 2025/1416, 2025.
[5] D. Connolly, P. Schwabe, and B. Westerbaan, "X-Wing: The Hybrid KEM You've Been Looking For," draft-connolly-cfrg-xwing-kem (CFRG), 2024.
[6] D. J. Bernstein, K. Bhargavan, *et al.*, "KyberSlash: Exploiting Secret-Dependent Division Timings in Kyber Implementations," *IACR TCHES*, 2025.
[7] M. Bellare and V. T. Hoang, "Efficient Schemes for Committing Authenticated Encryption," *EUROCRYPT*, 2022.
[8] "On the Role of Public Contexts in Hybrid KEMs," IACR ePrint 2026/140.
[9] National Institute of Standards and Technology, "FIPS 204: Module-Lattice-Based Digital Signature Standard," 2024.
[10] National Institute of Standards and Technology, "FIPS 205: Stateless Hash-Based Digital Signature Standard," 2024.
[11] J. Bos *et al.*, "CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM," *IEEE EuroS&P*, 2018.
[12] G. Alagic *et al.*, "Status Report on the Fourth Round of the NIST Post-Quantum Cryptography Standardization Process," NIST IR 8545, 2025.
[13] E. Fujisaki and T. Okamoto, "Secure Integration of Asymmetric and Symmetric Encryption Schemes," *CRYPTO*, 1999.
[14] D. Hofheinz, K. Hövelmanns, and E. Kiltz, "A Modular Analysis of the Fujisaki–Okamoto Transformation," *TCC*, 2017.
[15] A. Langley, M. Hamburg, and S. Turner, "Elliptic Curves for Security," RFC 7748, IETF, 2016.
[16] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, 2018.
[17] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid Key Exchange in TLS 1.3," draft-ietf-tls-hybrid-design, IETF, 2023.
[18] K. Kwiatkowski *et al.*, "Post-Quantum Key Exchange `X25519MLKEM768` for TLS 1.3," draft-kwiatkowski-tls-ecdhe-mlkem, IETF, 2024.
[19] K. Bhargavan, C. Jacomme, F. Kiefer, and R. Schmidt, "Formal Verification of the PQXDH Post-Quantum Key Agreement Protocol," *IEEE S&P*, 2024.
[20] Apple Security Engineering, "iMessage with PQ3: The New State of the Art in Quantum-Secure Messaging," Apple Security Research, 2024.
[21] K. Hövelmanns *et al.*, "Binding Security of Implicitly-Rejecting KEMs and Application to BIKE and HQC," IACR ePrint 2024/1233, 2024.
[22] J.-K. Zinzindohoué, K. Bhargavan, J. Protzenko, and B. Beurdouche, "HACL*: A Verified Modern Cryptographic Library," *ACM CCS*, 2017.
[23] Cryspen, "Verified ML-KEM (Kyber) in `libcrux`," 2024.
[24] J. B. Almeida *et al.*, "Formally Verifying Kyber: Episode V," *CRYPTO*, 2024.
[25] G. Barthe, B. Grégoire, S. Heraud, and S. Z. Béguelin, "Computer-Aided Security Proofs for the Working Cryptographer," *CRYPTO*, 2011.
[26] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," *CAV*, 2013.
[27] B. Blanchet, "Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif," *Found. Trends Priv. Secur.*, 2016.

[28] O. Reparaz, J. Balasch, and I. Verbauwhede, “Dude, is My Code Constant Time?” *DATE*, 2017.
[29] N. Nethercote and J. Seward, “Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation,” *ACM PLDI*, 2007.
[30] A. Langley, “ctgrind: Checking that Functions are Constant Time with Valgrind,” 2010.
[31] National Institute of Standards and Technology, “Automated Cryptographic Validation Protocol (ACVP),” https://github.com/usnistgov/ACVP, 2024.
[32] J. Birr-Pixton *et al.*, “rustls: A Modern TLS Library in Rust,” https://github.com/rustls/rustls, 2024.

Appendix A
Artifact Availability

The complete suite—Rust core and backends, the five language faces, the EasyCrypt/Tamarin/ProVerif developments, the constant-time and netem harnesses, and the figure-generation scripts—is open source at https://github.com/billlza/q-periapt. All numbers in this paper are reproducible from that repository; the bare-metal netem protocol is provided as a script.

Appendix B
The Machine-Checked Kernel in Detail

Table V inventories the EasyCrypt development. Of the named results, all are fully discharged (no `admit`/`conjecture`); the only assumptions are the two elementary `be8_*` facts about the 8-byte length field and the modeling of H as collision-resistant. We validate that the proof is non-vacuous by an *adversarial negative control*: for each binding theorem we mechanically delete a single hint from its closing tactic and re-run the checker, confirming the proof then *fails* (`cannot prove goal`). Deleting `encode_inj` breaks every binding theorem; deleting the disagreement lemma (`ct_neq_fields_neq`, etc.) breaks the corresponding axis; and—in the explicit-rejection game—deleting the $K \neq \perp$ conjunct from the predicate makes the theorem unprovable, since two mutually-rejecting executions would otherwise “win” without inducing a hash collision. This last control is what distinguishes a faithful Figure-6 instantiation from a vacuous one.

TABLE V
EasyCrypt lemma inventory (`BindingViaCR.ec`). All proved; assumptions: `be8_*`, CR(H).

Lemma	Statement (informal)
`lp_cat_inj`	length-prefixed fields are self-delimiting
`encode_inj`	the field encoding is injective
`bind_le_cr`	generic transcript-collision $\leq$ CR(H)
`bind_le_cr_k{ct,pk,ctx}`	instantiated projections (CT/PK/CTX)
`malbind_k{ct,pk,ctx}_le_cr`	KEM-aware, implicit rejection $\leq$ CR(H)
`malbind_*_xrej_le_cr`	full Fig. 6, explicit rejection, $K \neq \perp$

Appendix C
Extended Evaluation

Table VI gives the microbenchmarks of Table IV with Criterion's 95% confidence intervals (arm64 host). Table VII gives the jitter/loss data of Section VI: the median is loss-insensitive, while the tail is governed by a per-loss TCP retransmission timeout and is, at 150 samples, statistically indistinguishable across the three groups—hence we report it as “RTO-dominated, transport-bound” rather than as a per-group claim. Footprint: the stripped C-ABI library is 441 KB (entire suite), the WebAssembly module 249 KB.

TABLE VI
Microbenchmarks, μs [95% CI]. arm64 host, Criterion.

Scheme	keygen	encaps	decaps
ML-KEM-512	6.6 [6.6,6.7]	6.1 [5.9,6.5]	6.8 [6.8,6.9]
ML-KEM-768	11.3 [11.2,11.4]	11.0 [10.9,11.0]	11.8 [11.7,11.8]
ML-KEM-1024	18.5 [18.1,19.2]	16.5 [15.8,17.7]	16.7 [16.5,16.8]
X25519	44.1 [43.8,44.4]	89.6 [89.2,90.0]	45.0 [44.7,45.3]
Hybrid, ContextBound	—	108.4 [107.9,108.9]	65.4 [65.1,65.7]
Hybrid, CompatXWing	—	102.2 [101.4,103.0]	59.2

TABLE VII
Time-to-session under jitter/loss, p50/p99 (μs). The p99 under loss is RTO-dominated and statistically indistinguishable across groups.

netem (RTT 20 ms +)	X25519	ContextBound	CompatXWing
+3 ms jitter	41.5k / 49.9k	41.6k / 50.1k	41.2k / 49.5k
+1% loss	41.1k / ∼1.08M	41.3k / (noisy)	41.4k / ∼1.09M
+3% loss	41.2k / ∼1.09M	41.6k / ∼1.09M	41.5k / ∼1.10M

Appendix D
Reproduction

Every result is reproducible from the artifact. The EasyCrypt kernel: `make -C formal/easycrypt check`. The constant-time discriminator: `sh ctstats/scripts/ct-gap-probe.sh` (Linux+Valgrind; ML-KEM and HQC). The binding/key-format demonstration: `cargo test -p q-periapt-backends --test binding_keyformat_separation`. Microbenchmarks: `cargo bench -p q-periapt-backends`. The TLS netem evaluation: `paper/netem-camera-ready.sh` (bare metal). Cross-substrate byte-identity: the per-face vector tests; the symbolic models: `make` under `formal/{tamarin,proverif}`. The figures of this paper rebuild via `make -C paper/figures`.

Acknowledgment

The author thanks the maintainers of libcrux/HACL*, EasyCrypt, Tamarin, and ProVerif, whose tools this work builds upon. *[Funding/advisor acknowledgments to be added.]*